AI COLLAB STUDIO / PROJECT BRIEF

Think together.
Build with AI.

A collaborative coding workspace where teams can discuss ideas in real time, ask an AI assistant for development help, edit generated project files, and preview a running app in the browser.

A shared room for the whole build process

Create a project, invite collaborators, work through ideas in a live conversation, and move from an AI-generated file tree into an editable, runnable web project.

What the application does

01 Organize

Sign up, create a named project, and see projects connected to your account.

02 Collaborate

Invite registered users and exchange project messages through Socket.IO.

03 Ask AI

Mention @ai in chat to request development help from Google Gemini.

04 Build & preview

Browse generated files, edit code, run it in a browser WebContainer, and preview the result.

Created by

Atique Hussain Rangrej, Ashish Patidar, and Ashish Nayak.

A practical shared workspace for coordinating in chat while using a common project file tree to code and preview.

HOW THE PIECES WORK TOGETHER

Frontend, backend & database

The browser interface handles the collaborative workspace and coding experience. The backend authenticates users, manages project data, coordinates live events, and calls the AI service. MongoDB holds durable account and project records.

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend	React 18, Vite, Tailwind CSS, React Router	Renders dashboard, authentication, project chat, collaborator selection, file explorer and editor. Axios calls REST endpoints; Socket.IO client joins live project rooms.
In-browser runtime	WebContainer API	Mounts the project file tree in the browser, runs install/start commands, and provides a local preview URL for the embedded app.
Backend	Node.js, Express, Socket.IO	Exposes user, project and AI routes. JWT checks protect API calls; bcrypt hashes passwords. Socket.IO broadcasts project chat and orchestrates @ai replies.
Database	MongoDB with Mongoose	Stores users (email and password hash) and projects (name, collaborators and fileTree). Project edits are persisted through the update-file-tree endpoint.
Supporting services	Redis, Google Gemini	Redis checks revoked JWTs during authentication. Gemini generates AI responses; its key is read from GOOGLE_AI_KEY.

Typical request flow

1. **Sign in** — The frontend sends credentials to Express. The server verifies the password and issues a JWT.
2. **Open a project** — REST endpoints fetch project details; the frontend connects its socket to that project room using the token.
3. **Collaborate** — Chat messages are broadcast to room members. Messages containing @ai are sent to Gemini and the AI response is returned to the room.
4. **Edit & run** — The file tree is mounted in WebContainer. Code edits are sent to Express and stored in MongoDB; the runtime starts the app for preview.

IMPLEMENTATION NOTES

Data, services & setup

A quick guide to the current implementation and the environment it expects.

What is stored where

• MongoDB: user accounts and project documents. A project contains its name, collaborator references, and generated or edited file tree.

• Redis: revoked-token lookup used by the authentication middleware. It is part of the request authentication path.

• Socket.IO: live project messages are passed between connected collaborators. The current server does not persist chat history to MongoDB, so messages are live-event data.

• WebContainer: project files are mounted and run in the browser environment for an interactive preview; this is separate from the Express API server.

• Gemini: @ai prompts are sent to the configured Google Generative AI model. The model is instructed to return structured text and, where applicable, a file tree.

Configuration

VARIABLE	USED BY	PURPOSE
MONGODB_URI	Backend	MongoDB connection string.
JWT_SECRET	Backend	Signs and verifies authentication tokens.
GOOGLE_AI_KEY	AI service	API key for Google Generative AI.
REDIS_HOST / REDIS_PORT / REDIS_PASSWORD	Redis client	Redis connection settings for revoked-token checks.
PORT	Backend	API and Socket.IO server port; defaults to 3000.

Run locally

Start the backend and frontend in separate terminals. Configure the backend environment variables above, start MongoDB and Redis, then run the Vite development server. The frontend API base URL is configured in frontend/src/config/axios.js.

Current product boundaries

AI responses depend on valid Google AI credentials and network access. The project editor persists file-tree changes, while chat history is not currently stored. The browser-based preview depends on WebContainer support in the browser.

Project creators
Atique Hussain Rangrej | Ashish Patidar | Ashish Nayak